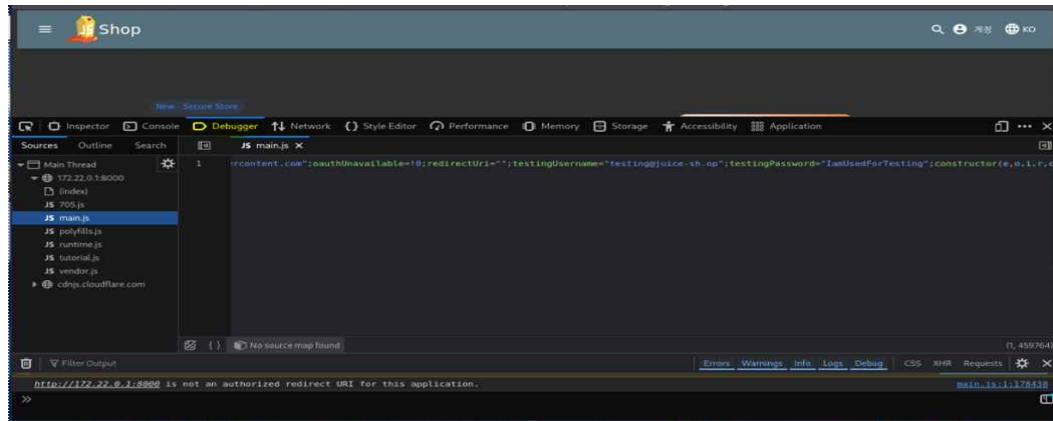


02	Expose Credential (자격 증명 노출) 취약점 개요
점검 내용	웹사이트의 하드코딩이 되어있는지 확인
점검 목적	웹사이트 내 인가되지 않은 접근 차단을 위해 코드 내에 포함된 ID/비밀번호, API 키 등을 추출해 데이터베이스나 클라우드 서버에 무단 접속하는 것을 방지함
보안 위협	하드코딩(Hardcoding)된 자격 증명을 점검하는 이유는 소스 코드나 설정 파일에 직접 기재된 민감 정보가 외부에 노출될 경우, 공격자가 별도의 해킹 기술 없이도 시스템의 정당한 권한을 통째로 탈취함
참고	- Expose Credential이란 소스코드, 설정 파일, 로그, 클라이언트 코드 등에 인증 정보(Credential)가 평문으로 노출되는 보안 취약점을 의미한다. - 하드 코딩(Hard Coding)이란 데이터나 설정을 코드 안에 직접 고정값으로 박아넣는 프로그래밍 방식으로, 외부 입력이나 설정 파일 없이 소스 코드 자체를 수정해야만 변경할 수 있어 유연성과 확장성이 떨어지지만, 코드가 직관적이고 빠른 장점이 있어 절대 변하지 않을 값에 한정적으로 사용되기도 합니다
점검대상 및 판단기준	
대상	웹 애플리케이션 소스 코드 확인
조치방법	웹 사이트의 게시판, 1:1 문의, URL 등에서 사용자 입력 값에 대해 검증 로직을 환경 변수를 사용하여 소스코드에 민감 정보 미포함 시킴 또는 secret Manager 사용하여 암호화 저장 혹은 AWS 사용시 IAM 접근 제어
점검 및 조치사례	
1. 홈페이지의 소스코드를 보기위해 F12를 입력하여 구조를 파악합니다.	

<head>,<body>를 보면 <script src="main.js" type="module"></script>이 존재한다.

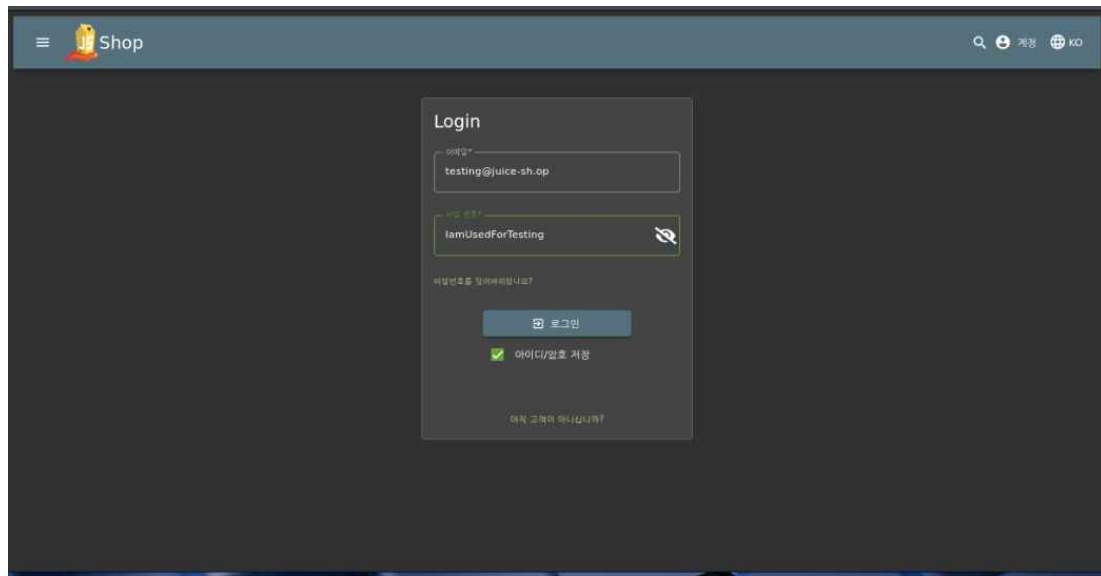
2. Debugger에서 파일의 구조 및 코드를 파악한다.



코드를 보던 중 하드 코딩이 많이 되어 있음을 확인 코드 상에서 pw, user, username, password, pwd 등 아이디와 비번을 상징하는 것을 검색해서 찾아 본다.

```
//main.js
//..... 앞부분 생략
class n {
  //... 중간생략
  emailControl = new l.hs("", [l.k0.required]);
  passwordControl = new l.hs("", [l.k0.required, l.k0.minLength(1)]);
  hide = !0;
  user:
  rememberMe = new l.hs(!1);
  error:
  clientId =
  "1005568560502-6hm16lef8oh46hr2d98vf2ohlnj4nfhq.apps.googleusercontent.com";
  oauthUnavailable = !0;
  redirectUri = "";
  testingUsername = "testing@juice-sh.op";
  testingPassword = "IamUsedForTesting";
  constructor(e, o, i, r, c, p, k, x) {
  // ... 뒷부분 생략 ...
```

3. Account login에 입력하여 login후 정보 탐색 및 침투하여 권한을 탈취한다.



4. 보안 방법

1. 개발 중 사용한 비번과 유저아이디 부분을 삭제한다.

```
1005568560502-6hm16lef8oh46hr2d98vf2ohlnj4nfhq.apps.googleusercontent.com";
oauthUnavailable = !0;
redirectUri = "";
testingUsername = "testing@juice-sh.op"; //삭제
testingPassword = "IamUsedForTesting"; //삭제
constructor(e, o, i, r, c, p, k, x) {
// ... 뒷부분 생략 ...
```

2. Credential 외부화 (환경 변수 사용)

```
export DB_PASSWORD=seccure_password
string password =System.getenv("DB_PASSWORD");
```

3. Credential 최소 권한 원칙 적용

DB 계정 권한 최소화
API Key 사용 범위 제한
관리자 계정 사용 금지

4. 정적 분석 및 자동 점검 도구 활용

※ 커밋 단계에서 Credential 노출 탐지

Tool
GitHub Secret Scanning
TruffleHog
Gitleaks
SonarQube